Promo Finder: Ranked Promo Clips from Microdrama Screenplays

Solution Document | Problem 1: Promo Finder | Repository: github.com/chandra8688/kuku-promo-finder

Result in brief. Of the 25 human-labelled promo windows in the dev set, the pipeline finds 24; the "last 45 seconds of each episode" baseline finds 12 (the weights were fitted on this set, so this is in-sample). On the 7-window reserved test set it finds 6 to 7, against the baseline's 5. A person reading the screenplay approved 13 of the 15 promos it ranks first across the show. It only ties the baseline at picking each episode's single best clip, because these episodes end on cliffhangers.

1. Problem Selected & Why

We picked **Problem 1, Promo Finder**. Vertical microdramas live on short promo clips (20 to 60 s) for Reels, Shorts and TikTok, and finding them means an editor scrubbing 50 to 100 episodes per show. Working from the screenplay means a ranked shortlist exists before the footage is cut, so the editor starts from candidates instead of a blank timeline.

2. Core Assumptions

- **Hook in 3 seconds.** The opening 3 s must carry a conflict, a provocative line, a loud SFX or a physical action.
- **Ends unresolved.** The final beat leaves an open question; a clip whose tension resolves is not a promo.
- **One continuous span.** No stitching across episodes. A span may cross a scene boundary inside an episode; such crossings are flagged for the reviewer, not blocked.
- **Screenplay, not footage.** The system scores screenplay promo potential. Acting, shot quality, music and edit rhythm are invisible to it, and nothing here measures click-through or retention.
- **The deliverable is show-level.** The question is which promos to post first across the whole show, so ranking is global with at most 2 clips per episode.

3. How the Solution Works

The key design choice. A model reads each episode once and scores every beat; code then searches every possible 20 to 60 s window and ranks them by those scores. The search is exhaustive, deterministic and costs no model calls, and the model never writes a timestamp, so clip boundaries cannot be hallucinated. Comparing windows pairwise with a model was estimated at 22,350 calls for 50 episodes.

- **S1 Parse** (`src/parse.py`, `src/speaker.py`): turns `.txt` screenplays into beats (DIALOGUE, ACTION, SFX, CAMERA) with centisecond timings; handles speaker headers with several parentheticals, which previously dropped the emotion label on 56 beats across 24 episodes.
- **S2 Annotate** (`src/annotate.py`): one model call per episode, whole episode in context, scoring each beat for hook (h), open question (q), resolution (r), context need (n) and intensity (t), returned by beat ID. Beats the model fails to score are counted as missing, not as zero, and make any clip containing them low-confidence.
- **S3-S4 Candidate engine** (`src/engine.py`): scores the whole span of each window: strongest hook in the clip's own first 3 s, strongest open question in the last 6 s, resolution, context need, duration-weighted intensity and position. SFX, action and camera beats can carry the hook through a hand-written list of cue words (a slap, a crash zoom). Hard gates are structural only; narrative weakness lowers the score instead of deleting the clip. `run.py`, the evaluation and the weight fit all call this one module with one config, `config/scoring_v2.yaml`, whose hash is written into every run.
- **S5 Judge** (`src/judge.py`): writes each promo's one-line "why", and removes a clip only when it and S2 both say the tension resolves (13 clips in the 50-episode run, listed in `judge_rejected.json`). Its scores do not affect ranking: in an earlier version of the pipeline, folding them in cut test top-1 from 5/5 to 2/5.
- **S6 Rank** (`run.py`): top 15 by S3 score, at most 2 per episode, low-confidence clips last, ties broken down to candidate ID.
- **S7 Refine** (`src/refine.py`): after the ranking is frozen, trims dead lead-in and ends the cut just after the last dialogue, SFX or action beat. Every adjustment is re-checked against the duration limits and the rule that no cut may slice through a line.

Example output, the top-ranked promo of the shipped run (`promos_ranked.json`): `episode-015`, 00:44.80 to 01:26.60 (41.8 s); judge scores hook 2/3, cliffhanger 2/3, cold-viewer clarity 2/3. Each promo also carries its hook line, cliffhanger line and one-line "why"; those are screenplay text, so they are not reproduced here.

4. How It Scales to Every Show

- **Linear call count.** One annotation call per episode plus batched judge calls. Measured on the 50-episode run: 100 model calls, 379,640 input tokens, 410.0 s wall clock (`run_manifest.json`), so per episode 2 calls, about 7,600 input tokens and 8.2 s. A 100-episode show needs about 200 calls.
- **Runs without a key.** `--no-llm` runs end to end with 0 network calls. That mode is the Tier-0 heuristic (baseline B3 below), not the full pipeline.
- **Config, not code.** Weights and thresholds live in YAML and every candidate file records which config scored it. Not yet tested on genres other than this show.
- **Reproducible where it can be.** Parsing, search, gates, refinement and ranking give identical output for identical input and config. LLM annotation does not; see section 6.

5. How We Know It's Good

Top-1: an episode's first pick overlaps a reference window by at least half (IoU of at least 0.5). **Recall**: the share of reference windows matched by one of the top 5 picks per episode. Bracketed figures are 95% bootstrap intervals over episodes.

Reserved test set: 5 episodes, 7 reference windows. These windows were proposed and reviewed by a model and never verified by a person, so read the numbers as agreement with an AI-assisted reference. Two independent annotation runs of the same model and prompt are shown: run A, used to fit the weights, and run B, which produced the shipped promos. Code and weights are identical; only the model's beat annotations differ. The set has been evaluated five times: three earlier code versions, then the final code on runs A and B, with nothing tuned on it. All five are listed in `eval_report.md`.

System	Top-1	Recall	Med start err	Med end err
V2, annotation run A	4/5 [40, 100]	6/7 [62, 100]	5.55 s	4.98 s
V2, annotation run B (shipped)	5/5 [100, 100]	7/7 [100, 100]	7.10 s	0.60 s

System	Top-1	Recall	Med start err	Med end err
B1 last 45 s	5/5 [100, 100]	5/7 [56, 100]	7.50 s	5.45 s
B3 Tier-0 heuristic	2/5 [0, 80]	5/7 [56, 100]	5.20 s	6.10 s
B2 random, 20 seeds	1.3/5	4.9/7	n/a	n/a

Dev set: 12 episodes, 25 human-labelled windows. The weights were fitted on this set, so these numbers are in-sample and favour V2 over the untuned baselines.

	V2 run A	V2 run B	B1 last 45 s	B3 Tier-0
Top-1	11/12	11/12	12/12	6/12
Recall	24/25	24/25	12/25	21/25
Med start err	4.45 s	5.35 s	4.15 s	3.00 s

Show-level precision@15: 13 of 15 approved [67, 100], labelled by Chandra Shekar (the author) in one pass, reading each clip's screenplay beats with the model's scores hidden (`scripts/review.py`). One labeller who built the system is a bias, and there is no baseline figure to compare against yet: B1 has no score, so it cannot rank across the show, and a labelled sample of B1's clips is the missing comparison.

6. Where It Fails

- **B1 ties us on per-episode top-1**, because writers put the cliffhanger at the end of every episode. Our case rests on mid-episode promos B1 never returns and on the show-level ranking it cannot make.
- **Start boundaries are the weakest number.** A hook in the first 3 seconds depends on where the clip starts, and the shipped run's median start error is 7.10 s on the test set (B1: 7.50 s) and 5.35 s on the dev set (B1: 4.15 s, Tier-0: 3.00 s). Refinement fixed the end; the start still lands seconds from where the reference cut is.
- **The evidence is thin.** Five test episodes cannot separate systems, and the test references are AI-reviewed.
- **Annotation jitter.** Two annotation runs of all 50 episodes, same model and prompt, agreed on 67% of beats for hook, 80% for open question, 64% for context need, 83% for intensity and 89% for resolution. The top-1 clip was the same clip (IoU at least 0.5) in 39 of 50 episodes. The test-set gap between runs A and B is this noise, not a change to the system. The shipped runner annotates once; a k-sample aggregation helper exists (`src/consistency.py`) but is not wired in.
- **Small-sample fitting.** An unconstrained fit on 25 windows flipped penalty signs and rewarded resolved endings. The fit is now bounded: penalties stay at or below 0, hook and cliffhanger at or above 1.0.
- **Pipeline drift, fixed.** `run.py` once scored with a different config from the one evaluated; a regression test now pins production, evaluation and tuning to one engine and one config.

7. Next Steps

- **Annotate each episode three times and take the median.** The 67% hook agreement between runs is the largest error source measured here, and the aggregation code already exists.
- **Anchor each clip's start on its hook beat** rather than the first beat of the window, to bring the start error down.
- **Shrink the judge**: write the "why" for the final 15 only, and spend the saved calls on a blind cold-viewer check of clarity, the brief's least-covered criterion.
- **Firmer evidence**: a person verifies the 7 test references, and a second labeller rates the top 15 alongside a sample of B1's clips.

Integrity: the test suite (`python -m pytest tests/ -q`), the leak check (`python scripts/leak_check.py`) and the submission check (`python scripts/check_submission.py`) all pass.